

Signed Recurrent Circuits for Chess Decisions: A Small Controlled Distillation Study

OpenJev project

Development report, September 2026

Abstract

We specify a bounded study of synthetic signed recurrent circuits for chess move selection. A structured circuit and a degree-preserving rewired control share training data, initialization of common modules, optimizer, and internal update count. A GRU provides a conventional baseline. Students distill fixed-node Stockfish decisions from generated, game-separated positions. A separate arena compares these students with published ChessFly and ChessLFM policies, an unchanged OpenJev Qwen scorer, and a Codex-dispatched Astra player. These external comparisons are heterogeneous demonstrations, not architectural ablations. Across three seeds on 1,024 development positions, circuit, rewired and GRU students achieve 14.3555%, 14.1927% and 14.5182% mean teacher agreement. The circuit’s 0.16276-percentage-point gain over rewired fails the predeclared 3-point continuation threshold. **This is a negative topology-screening result, not evidence of architectural superiority, Elo or a novel algorithm.** The 24-position public puzzle panel and 18-game exhibition schedule are complete. We propose a separate history-dependent state-transition diagnostic and distinguish it from computation within a board and biological connectome evidence.

1 Question and prior work

Does a structured signed graph improve chess decisions over a rewired graph when signed degrees, training data, and internal computation budget are held fixed? This question is narrower than whether recurrent networks can play chess. Designed sparse recurrent wiring is established in neural circuit policies [1]; policy distillation is established [2]; and recurrent computation with additional inference steps has already been studied on chess [3]. Our cell uses simple discrete tanh dynamics, not the continuous-time dynamics of neural circuit policies. Published ChessFly and ChessLFM artifacts provide further direct prior context [6, 7]. We do not claim novelty from combining these ingredients or placing them behind an all-legal-move interface.

2 Current student protocol

2.1 Observations, recurrence, and actions

A student receives an 841-dimensional numeric representation of the current FEN: 12 piece channels over 64 squares, side to move, castling rights, en-passant square including an explicit absent category, and two move counters. Teacher moves and scores never enter these features. A shared affine/tanh encoder produces $z \in \mathbb{R}^{64}$. Each decision starts from $h^{(0)} = 0$ and applies four updates to the *same board*. The GRU update is $h^{(k+1)} = \text{GRU}(z, h^{(k)})$. For either circuit arm,

$$h_i^{(k+1)} = \tanh\left(b_i + d_i z_i + \sum_{j:(j,i) \in E} w_{ij} h_j^{(k)}\right), \quad (1)$$

$$w_{ij} = \frac{s_{ij} \text{softplus}(a_{ij})}{\sqrt{\max(1, \deg_{\text{in}}(i))}}, \quad d_i = \mathbf{1}[i < 16]. \quad (2)$$

The structured graph has 16 sensory, 24 inter, 16 command, and 8 motor units, with 352 edges. Sensory units project to inter units; inter units project to command units; command units project to other command units and motor units. Every fourth source unit is inhibitory. The control uses a fixed 20,000 double-edge-swap proposal budget and preserves each node’s signed incoming and outgoing degrees, with no self-edges or duplicates. There is currently one structured graph and one rewired graph, not a graph-seed distribution.

The final hidden state feeds a 1,968-entry geometric UCI move head and a scalar tanh value head. The move vocabulary includes promotions; a python-chess mask retains *all* legal moves before softmax. Thus legality is supplied by a symbolic rules engine. The model is not being tested on learning chess rules. The value head reads all hidden coordinates, as does the move head; motor-unit labels do not imply a motor-only readout.

Capacity and implementation limits. The two circuit arms have 182,289 declared parameters each; the GRU has 206,833. Shared encoder and output-head initialization match within a weight seed. However, the circuits receive only 16 encoder coordinates whereas GRU receives all 64. Consequently 48 encoder rows, totaling 40,416 parameters, do not affect circuit outputs. The GRU comparison does not isolate either topology or input capacity. The circuit implementation builds a dense 64×64 matrix for each update; sparse parameterization is not evidence of sparse-kernel speed.

2.2 Teacher data and optimization

The protocol specifies 4,096 training and 1,024 development positions from disjoint generated games. At each step, the generator chooses a uniformly random legal move or the teacher move with equal probability. Each game has a 40-ply cap, with fixed generation caps of 160 training and 64 development games. Game seeds are predetermined. Global state deduplication includes pieces, side, castling and en-passant state, while ignoring halfmove/fullmove counters. Reaching a generation cap without filling the quota is a recorded failure, not a reason to extend the budget.

Stockfish 19 [8] labels positions with a 2,000-node request, one thread, 16 MB hash, and cleared hash before analysis. Teacher analysis receives FEN without preceding move history, matching the student’s history-free observation. Both labeling calls and teacher-selected rollout moves are charged to teacher computation; requested and reported nodes are retained separately. The target move is the first principal-variation move. The value target is $y_v = \tanh(c/600)$ for side-to-move centipawns c , using signed 10,000 centipawns for mate scores. This target is not a win probability.

Each of the three modes uses seeds 17, 29 and 43. Data, epoch permutations within seed, and training budget match across modes. Training uses three epochs, batch size 64, Adam with learning rate 10^{-3} , betas (0.9, 0.999), $\epsilon = 10^{-8}$, no weight decay, and gradient norm clipping at 1. The loss is

$$\mathcal{L} = -\log \pi_\theta(a^* \mid x, \mathcal{A}_{\text{legal}}(x)) + 0.5(v_\theta(x) - y_v)^2. \quad (3)$$

This gives 192 optimizer updates per fit and nine final fits. Runs use two CPU threads, randomized fit order, and no early stopping, checkpoint selection or replacement fits. This is supervised distillation, not reinforcement learning.

2.3 Evaluation and evidence

All final checkpoints use the same generated development set. Report per-seed and aggregate teacher agreement, target negative log likelihood, value MAE and MSE, parameters, and training/evaluation time. Reference baselines are the exact uniform-legal expected agreement and the most frequent training target among legal moves. Teacher agreement measures imitation of a bounded engine, not optimal move quality. Positions from a game are correlated; position count is not an independent-sample count.

Plans bind source, dependency and engine hashes. Data receipts retain generated games, FENs, analyses, teacher costs, and split membership. Fit receipts bind checkpoints, logs, predictions and fixed update counts. These checks establish execution identity, not efficacy. All nine supervised fits completed. Table 1 reports the saved development results; these positions are not an untouched confirmation panel.

For seeds 17/29/43 respectively, agreement is 14.16%, 14.45%, and 14.45% for circuit, 14.55%, 14.06%, and 13.96% for rewired, and 14.45%, 14.45%, and 14.65% for GRU. Uniform-legal expected agreement is

Arm	Declared parameters	Mean agreement	Target NLL	Value MAE
Synthetic circuit	182,289	14.3555%	3.113410	0.578403
Rewired circuit	182,289	14.1927%	3.107788	0.578546
GRU	206,833	14.5182%	3.117682	0.577060

Table 1: Means across three weight seeds on the same 1,024 generated development positions. Parameters include unused encoder rows in the two circuit arms.

4.4997%; training-target-frequency agreement is 7.1289%. The circuit-versus-rewired difference changes sign across seeds. This small-data, short-training study establishes some teacher imitation but does not support a topology advantage. It is not a test of larger models or longer training.

The frozen descriptive gate requires at least a 3-point circuit mean agreement gain over rewired; at most a 3-point mean deficit to GRU; no paired seed more than 5 points below rewired; and at most 0.03 additional mean value MAE. The first check fails: observed gain is 0.16276 points. The other three pass, so the conjunction fails. We retain that stop decision instead of extending the run or promoting a topology claim.

3 Separate public-player arena

The separately fixed 18-game schedule uses a color-paired round robin among the circuit student, published ChessFly, ChessLFM and Astra through Codex, plus circuit pairs against OpenJev Qwen, GRU and rewired students. Student seed 17 and the first scheduled game’s replay were selected before training outcomes, without selecting a winning checkpoint or replay. All 18 scheduled games reached scored outcomes, with no model-call failures or unfinished games. Table 3 retains every scheduled pairing.

OpenJev/Qwen. The adapter pins `mlx-community/Qwen3-4B-Instruct-2507-4bit` at revision `50d427756c6b1b2fe0c0a10f67fbd1fc8e82c1b`. It serializes the board, recent moves and every legal candidate, then restricts one next-token forward pass to tokenizer-verified labels. It generates no reasoning trace.

ChessLFM. The published 230M checkpoint is pinned at revision `341bfd2ea4b696cda7d2016d4a721f27eb9cf34d`. Its documented protocol uses an 80-token board representation, a predicted value token, then a legal move token from a second forward pass [7]. The adapter omits search. Its convolution/attention hybrid is not our learned recurrent world model.

ChessFly. Published weights are pinned at revision `ed6744ec984ba401f06d6b0a966a2661b1458f53`; graph assets come from pinned Space revision `375374bf58f1c828ad7d33905b64f1fa6ac4906f`. The adapter performs five settling steps from zero per board, without the browser demo’s search. Mathematical adaptation does not establish bitwise parity with its browser implementation. External weights and graph assets retain their upstream terms and are not relicensed by this project.

Astra via Codex. A separately selected `gpt-6-astra` agent receives only a board packet and legal candidates. The file bridge records request IDs, responses, timeouts and closure. Provider identity is not attested by the bridge itself. One persistent agent serves all puzzle and game requests, retaining earlier packet context but receiving no gold labels or score files. This is not an independently reset API request per move. Packets do not supply the remaining game clocks. Isolation relies on instructions, not an OS sandbox. It requests one move, not a fabricated probability distribution.

These systems differ in pretraining, parameter count, representations, search and execution route. The text packets for Astra and Qwen also contain SAN check and checkmate markers computed by `python-chess`. Three of the 24 puzzle positions have an immediate mate revealed by this notation. Numerical policies receive legal masks but no SAN labels. This assistance is a further confound; the panel does not measure unaided chess reasoning. Their candidate softmax scores are uncalibrated legal-move preferences, not probabilities of winning. No engine targets are supplied to the learned policies during play. Rules, histories and accepted

Policy	Strict first-move matches / 24
Stockfish, 1,000 nodes	22
GPT-6 Astra via Codex	17
ChessLFM direct	14
ChessFly direct	7
Greedy material	5
OpenJev circuit, seed 17	2
OpenJev rewired, seed 17	2
OpenJev Qwen direct	2
OpenJev GRU, seed 17	1
Seeded uniform random	1

Table 2: Public convenience-prefix development panel, not full-puzzle solves, an unseen-data claim, a representative benchmark or an Elo estimate.

moves remain under the authoritative chess engine; policy exceptions and illegal responses do not trigger hidden fallbacks.

The configurable protocol defaults to 300-second clocks per side, no increment, and a 120-ply cap. A capped game is unfinished, never automatically a draw. Automatic chess outcomes, policy/transport failures and flag falls are recorded separately. An external request deadline shorter than the remaining game clock can produce a failed game rather than a chess loss. Initial loading and declared warmups are excluded from game clocks; policy wall time is charged. Astra wall time includes dispatch, reasoning, tool use and polling. It is not comparable to a local kernel-only inference measurement. A few games cannot estimate Elo or reproduce a third-party tournament.

The completed public puzzle panel takes the first eight valid records per rating band (≤ 1200 , 1201–1800, > 1800) from a bounded official Lichess CSV prefix [9]. It applies the first recorded opponent move before scoring the first solver move. Strict gold agreement and an immediate-mate alternative are separate metrics; neither is a full-puzzle solve rate. These 24 convenience positions are development data, not a representative benchmark or an assuredly uncontaminated test for published models.

All 240 puzzle-policy calls completed without errors. Table 2 reports strict first-move matches from the saved call records. Differences in training exposure and compute prevent causal architecture or chess-rating interpretations. The complete game outcomes are reported separately below.

3.1 Complete game outcomes

Astra won four of its six games and lost two: a clock loss to ChessFly and a checkmate loss to ChessLFM. ChessLFM and Astra split their two games. The circuit had zero wins, seven draws and five losses across its twelve scheduled games. Seven games ended by automatic fivefold repetition, one by stalemate, nine by checkmate and one by timeout. No outcome was assigned from a centipawn evaluation. These are descriptive route outcomes from one starting position, not a rating or a biological-topology effect. In particular, the clock loss includes Codex overhead and the agent did not receive the remaining clock in its packets.

4 Three claims that must remain separate

Computation within a position. The implemented recurrence refines one observed board. Resetting hidden state between boards is part of the model. Extra internal steps are not additional environment transitions, a search tree, or memory across moves.

Biological wiring. Our 64-unit graph is synthetic. Its sensory/inter/command/motor names do not import a biological connectome. Separately running pretrained ChessFly cannot isolate a causal effect of its graph; that would require common-data retraining with biological, rewired and synthetic graph controls.

Pairing (first player perspective)	Wins	Draws	Losses
Circuit – ChessFly	0	1	1
Circuit – ChessLFM	0	0	2
Circuit – Astra	0	0	2
ChessFly – ChessLFM	0	1	1
ChessFly – Astra	1	0	1
ChessLFM – Astra	1	0	1
Circuit – Qwen	0	2	0
Circuit – GRU	0	2	0
Circuit – Rewired	0	2	0

Table 3: Every color-paired exhibition. The ChessFly win against Astra was on clock; all other decisive games ended by checkmate. Zero capped or failed games. Different opponents, observations, training and compute prohibit an aggregate architecture leaderboard.

Recurrent world models. A world model requires learned, action-conditioned future-state prediction, with appropriate multi-step evaluation; latent imagination adds another behavior-learning question [4]. These students have neither. Standard chess supplies an exact transition engine, so a learned world model would additionally need to justify its utility against that available reference.

5 Proposed memory and state-transition diagnostic

This section is not frozen or run, and does not rescue the failed topology gate. We propose testing persistent predictive state on information absent from the current FEN. Repetition is a concrete target with an exact reference [10]. From the standard board, repeating `g1f3 g8f6 f3g1 f6g8` twice and playing `g1f3 g8f6 b1c3 b8c6 c3b1 c6b8 f3g1 f6g8` give identical full FENs, including counters, and identical final moves. Only the first history has a third occurrence of the current position. This rule-level construction has been checked; it is not a learned-model result. The target is current-position threefold occurrence, not the broader possibility of claiming a draw on a subsequent move.

Generate balanced endpoint pairs across many legal prefixes and cycle families. Put both pair members and their whole prefix family in the same split. Use separate training, calibration and test game-seed panels; reject cross-split endpoint overlap while deliberately retaining identical endpoints within a pair. Freeze a longer-cycle/distractor shift before training. Counters, previous actions and other supplied features must match within each endpoint pair, so labels cannot be read from an incidental shortcut. A fixed generation cap and durable failure records prevent hand-picking successful constructions.

Compare current-only state, a persistent 64-state GRU, and that GRU with an action-conditioned latent transition plus future-observation and repetition heads. Add a same-sized auxiliary reconstruction control. Match feature access, readout capacity, data, optimizer/update budgets and fresh seeds. None should inherit pilot student weights. Exact python-chess history replay is the reference; resetting learned state at every step is the causal memory intervention. This isolates memory use before attempting chess-playing RL.

Predict one-, two- and four-step futures from the inferred state and supplied actions, without later observations or future legal masks. Report exact board accuracy, changed-square accuracy, side/castling/enpassant/counter errors, invalid predicted states, repetition balanced accuracy, calibration and cost. Square-average accuracy alone can reward copying unchanged pieces. Ground-truth legality constrains data generation, not predicted rollout states.

A bounded proposed start is 4,096 training sequences, 512 calibration pairs and 512 test pairs per condition, three fresh seeds and 192 updates per arm. Sequence lengths, loss weights, transition-loss controls, runtime budget and precision must be frozen after generation/runtime checks that do not score the test panel. These counts are proposals, not completed work.

A diagnostic gate should require at least 90% repetition accuracy on both unseen-prefix and longer-history panels in every fit, a state-reset drop of at least 20 percentage points, and a frozen practical improvement in rollout quality over recurrent/reconstruction controls at matched compute. The balanced identical-input

deterministic current-only control has a 50% information ceiling; exceeding it indicates leakage or imbalance. Account for pair/game and training-seed variation. Passing would establish learnability and memory use, not playing strength or novelty.

An ICLR-oriented claim still requires a specific mechanism beyond standard recurrence/prediction, untouched transfer at matched compute, a strong explicit associative-memory baseline, and an intervention linking mechanism to outcome. If topology is revisited, require multiple structured and degree-preserving random graph seeds plus common-input capacity controls. The present graph claim remains stopped. Adaptive stopping would additionally need adaptive computation time [5], fixed-depth, entropy-gated and matched-budget ensemble controls, including gate cost. Conformal calibration should address a measured decision risk rather than replace these comparisons.

6 Status and reproducibility

The supervised development report, puzzle panel and game schedule are complete. The proposed state-transition study has not been frozen or run. The source protocols are `scripts/train_chess_student.py` and `scripts/chess_study.py`; model implementations are under `src/openjev/research/chess_*.py`. The saved supervised report is `evidence/chess-student-v1/results/report/summary.json`, bound to plan SHA-256 `b798fc8a421a6e72479085b990c8439b9069d915ff0cd8c888151ff440e70d45`. The separately fixed arena design is `evidence/chess-v1/design.json`. Puzzle calls are recorded in `evidence/chess-v1/results/puzzles.jsonl`. Prepared plans and completed receipts must accompany later result updates. This manuscript is standalone and does not revise previous Doom or memory-study findings. No claim of state-of-the-art performance is made.

References

- [1] M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, and R. Grosu. Neural circuit policies enabling auditable autonomy. *Nature Machine Intelligence*, 2:642–652, 2020. doi:10.1038/s42256-020-00237-3. Author implementation: <https://github.com/mlechner261/ncps>.
- [2] A. A. Rusu et al. Policy Distillation. 2015. <https://arxiv.org/abs/1511.06295>.
- [3] A. Schwarzschild et al. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. 2021. <https://arxiv.org/abs/2106.04537>.
- [4] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. ICLR, 2020. <https://arxiv.org/abs/1912.01603>.
- [5] A. Graves. Adaptive Computation Time for Recurrent Neural Networks. 2016. <https://arxiv.org/abs/1603.08983>.
- [6] M. Labonne. ChessFly model card and executable Space assets. <https://huggingface.co/mlabonne/chessfly/tree/ed6744ec984ba401f06d6b0a966a2661b1458f53>. <https://huggingface.co/spaces/mlabonne/chessfly/tree/375374bf58f1c828ad7d33905b64f1fa6ac4906f>.
- [7] M. Labonne. LFM2.5-230M-Chess model card and checkpoint. <https://huggingface.co/mlabonne/LFM2.5-230M-Chess/tree/341bfd2ea4b696cda7d2016d4a721f27eb9cf34d>.
- [8] Stockfish developers. Stockfish 19. https://github.com/official-stockfish/Stockfish/releases/tag/sf_19.
- [9] Lichess. Open database, chess puzzles, CC0. <https://database.lichess.org/#puzzles>.
- [10] python-chess developers. Core documentation, FEN and repetition predicates. <https://python-chess.readthedocs.io/en/latest/core.html>.